Exact SAT Solving and Empirical Evaluation for the Large-Scale N-Queens Problem: A Comparative Study with Constraint Programming and Integer Linear Programming

Le Trung Hieu

Faculty of Information Technology, VNU University of Engineering and Technology (VNU-UET), 144 Xuan Thuy, Cau Giay, Hanoi, 100000, Vietnam

ARTICLE INFO

Keywords:
N-Queens Problem
SAT Solving
PySAT
Constraint Programming
Integer Linear Programming
At-Most-One Encodings
Conflict-Driven Clause Learning

ABSTRACT

The N-Queens problem is a classical combinatorial benchmark in computer science that tests the limits of exact constraint-solving algorithms. Although classical backtracking struggles beyond moderate sizes ($N > 30$), modern solving paradigms offer exact solutions for significantly larger instances. In this paper, we develop a comprehensive exact SAT-based framework using PySAT to model and solve the large-scale N-Queens problem on board sizes up to $N = 100$ (10,000 Boolean decision variables and over 1.65 million clauses). To address the combinatorial clause explosion inherent in naive pairwise encodings, we systematically investigate five At-Most-One (AMO) encoding techniques: Pairwise ($O(k^2)$ clauses), Sequential Counter (Sinz 2005), Bitwise Binary encoding, Commander Variable (Klieber & Gao 2007), and 2-Product (Chen 2010). In addition, we conduct a cross-paradigm evaluation comparing four SAT solvers (MiniSat 2.2, CaDiCaL 1.5.3, Glucose 4.0, and Glucose 3.0) against Google OR-Tools CP-SAT (under both 1D all-different and 2D Boolean formulations) and 0-1 Integer Linear Programming (SCIP). Our extensive computational experiments reveal three key scientific findings: (1) SAT and CP-SAT scale with consistent efficiency, solving $N = 100$ in under 4 seconds, whereas ILP suffers a severe combinatorial breakdown and times out at $N = 100$ due to excessive fractional assignments in its linear programming relaxation; (2) compact AMO encodings such as 2-Product and Sequential Counter reduce the generated clause count by 92.9% compared to the pairwise encoding (from 1,646,900 to 116,636 clauses), substantially reducing memory overhead while maintaining rapid solver convergence; and (3) a clear encoding–solver interaction exists where propagation-strong CDCL solvers like MiniSat excel on compact encodings, whereas solvers with aggressive restart strategies show varied sensitivity to auxiliary variables.

1. Introduction

The N-Queens problem is a classic combinatorial puzzle in mathematics and computer science [2]. Originally proposed by chess composer Max Bezzel in 1848 for an 8×8 board and investigated by Carl Friedrich Gauss, the generalized N-Queens problem requires placing N non-attacking queens on an $N \times N$ chessboard such that no two queens share the same row, column, or diagonal.

While simple to state, finding a valid configuration of N queens is fundamentally a constraint satisfaction problem (CSP). The search space of placing N queens on N^2 squares contains $\binom{N^2}{N}$ possible arrangements, or N^N configurations when restricting to one queen per row. Traditional depth-first search with backtracking suffers from exponential time complexity ($O(N!)$), rendering naive approaches computationally intractable for board dimensions beyond $N \approx 30$.

Over the past two decades, advances in automated reasoning, Conflict-Driven Clause Learning (CDCL), and constraint programming have transformed practical combinatorial optimization, enabling exact engines to routinely solve industrial instances with millions of constraints [4]. Three predominant exact paradigms have emerged:

1. **Boolean Satisfiability (SAT)**: Translating constraints into Propositional Logic in Conjunctive Normal Form (CNF) and applying modern CDCL solvers [3, 1].
2. **Constraint Programming (CP)**: Formulating high-level declarative constraints (e.g., global `AllDifferent`) solved via domain filtering and lazy clause generation, exemplified by Google OR-Tools CP-SAT [9].

*Corresponding author. Student ID: 24021475. Email: 24021475@vnu.edu.vn
Email address: 24021475@vnu.edu.vn (L.T. Hieu)
ORCID(s):

3. **Integer Linear Programming (ILP/MIP)**: Expressing conditions as systems of linear inequalities over binary integer variables, solved via Branch-and-Bound and cutting planes using engines such as SCIP [6].

A central challenge when modeling N-Queens in propositional logic is the representation of cardinality constraints, specifically **At-Least-One (ALO)** and **At-Most-One (AMO)** restrictions. Naive pairwise AMO encodings generate $O(k^2)$ binary clauses for a set of k literals, resulting in an $O(N^3)$ clause explosion across all diagonal constraints. At $N = 100$, this yields over 1.6 million clauses, creating substantial memory overhead and slowing down clause database management.

To address these challenges, this study provides four principal contributions:

- **Comprehensive SAT Modeling with 5 AMO Encodings**: We implement modular Python encoders in PySAT [7] incorporating five distinct AMO formulations: Pairwise, Sequential Counter [10], Bitwise / Binary, Commander Variable [8], and 2-Product [5].
- **Multi-Solver SAT Evaluation**: We systematically benchmark multiple CDCL engines including MiniSat 2.2, CaDiCaL 1.5.3, Glucose 4.0, and Glucose 3.0 to explore solver-encoding synergies and sensitivity to auxiliary variables.
- **Cross-Paradigm Benchmark**: We compare SAT solvers against both CP-SAT (under 1D AllDifferent and 2D Boolean formulations) and 0-1 ILP (SCIP MIP) on board dimensions up to $N = 100$ (10,000 Boolean cells).
- **Empirical and Theoretical Analysis**: We explain the underlying mathematical factors governing solver behavior, demonstrating why ILP breaks down at large N due to unconstrained fractional LP relaxations, and showing how 2-Product encoding achieves a 92.9% reduction in formula size.

The remainder of this paper is structured as follows. Section 2 formalizes the N-Queens problem and its CP and ILP formulations. Section 3 details the proposed SAT encodings and AMO techniques. Section 4 presents the empirical results and discussion. Section 5 concludes the paper.

2. Problem Formulation and Alternative Exact Paradigms

2.1. Problem Definition

Let the chessboard be represented as an $N \times N$ grid with coordinates (r, c) where $0 \leq r < N$ denotes the row index and $0 \leq c < N$ denotes the column index. The goal is to determine an injective assignment of N queens to distinct grid positions such that:

$$\text{Row constraint:} \quad \forall r \in \{0, \dots, N-1\}, \text{ exactly one queen in row } r. \tag{1}$$

$$\text{Column constraint:} \quad \forall c \in \{0, \dots, N-1\}, \text{ exactly one queen in column } c. \tag{2}$$

$$\text{Diagonal constraints:} \quad \text{At most one queen on each falling diagonal } (r - c = k). \tag{3}$$

$$\text{At most one queen on each rising diagonal } (r + c = s). \tag{4}$$

For an $N \times N$ board, falling diagonals are indexed by $k \in \{-(N-2), \dots, N-2\}$, and rising diagonals (anti-diagonals) are indexed by $s \in \{1, \dots, 2N-3\}$.

2.2. Constraint Programming (CP) Formulation

In Constraint Programming, high-level structural constraints can be expressed concisely using global constraint primitives.

2.2.1. 1D AllDifferent Formulation (`CP-AllDiff`*)*

Let $Q_r \in \{0, \dots, N-1\}$ denote the column position of the queen in row r, for $r \in \{0, \dots, N-1\}$. The non-attacking requirements are formulated using three global `AllDifferent` constraints:

$$\texttt{AllDifferent}(Q_0, Q_1, \dots, Q_{N-1}), \tag{5}$$

$$\texttt{AllDifferent}(Q_0 - 0, Q_1 - 1, \dots, Q_{N-1} - (N-1)), \tag{6}$$

$$\texttt{AllDifferent}(Q_0 + 0, Q_1 + 1, \dots, Q_{N-1} + (N-1)). \tag{7}$$

This formulation introduces only N integer decision variables and 3 global constraints, allowing domain propagation algorithms (such as bounds and generalized arc consistency) to prune candidate assignments rapidly.

2.2.2. 2D Boolean Formulation (`CP-Boolean`*)*

Alternatively, CP-SAT can operate on a 2D Boolean grid $x_{r,c} \in \{0,1\}$, with linear constraints $\sum_{c=0}^{N-1} x_{r,c} = 1$ and $\sum_{r=0}^{N-1} x_{r,c} = 1$, alongside diagonal sum constraints $\sum_{(r,c)\in D} x_{r,c} \leq 1$.

2.3. Integer Linear Programming (ILP/MIP) Formulation

In ILP, the problem is modeled as a 0-1 binary integer linear program. For each row r and column c, define a binary variable:

$$x_{r,c} \in \{0,1\} \quad \forall r, c \in \{0,\dots,N-1\}, \tag{8}$$

where $x_{r,c} = 1$ if a queen occupies square (r,c), and 0 otherwise.

The ILP model is formulated as a feasibility problem (with an arbitrary constant objective):

$$\min \quad 0 \tag{9}$$

$$\text{s.t.} \quad \sum_{c=0}^{N-1} x_{r,c} = 1 \quad \forall r \in \{0,\dots,N-1\}, \tag{10}$$

$$\sum_{r=0}^{N-1} x_{r,c} = 1 \quad \forall c \in \{0,\dots,N-1\}, \tag{11}$$

$$\sum_{\substack{0\leq r,c<N \\ r-c=k}} x_{r,c} \leq 1 \quad \forall k \in \{-(N-2),\dots,N-2\}, \tag{12}$$

$$\sum_{\substack{0\leq r,c<N \\ r+c=s}} x_{r,c} \leq 1 \quad \forall s \in \{1,\dots,2N-3\}, \tag{13}$$

$$x_{r,c} \in \{0,1\} \quad \forall r, c \in \{0,\dots,N-1\}. \tag{14}$$

Theoretical Bottleneck of ILP on N-Queens: Standard MIP solvers (such as SCIP) rely on continuous Linear Programming (LP) relaxations within a Branch-and-Bound framework. In the LP relaxation of Equations (10)–(14), setting $x_{r,c} = \frac{1}{N}$ for all r, c yields:

$$\sum_{c=0}^{N-1} \frac{1}{N} = 1, \quad \sum_{r=0}^{N-1} \frac{1}{N} = 1, \quad \text{and} \quad \sum_{(r,c)\in D} \frac{1}{N} \leq \frac{|D|}{N} \leq 1. \tag{15}$$

Because the uniform fractional assignment $x_{r,c} = 1/N$ satisfies all row, column, and diagonal constraints with zero violation, the continuous LP relaxation provides virtually no pruning guidance to the dual simplex solver. Furthermore, because N-Queens is formulated as a pure 0-1 feasibility problem without an optimization objective ($c^T x = 0$), the LP dual bound remains identically zero at every subproblem. Consequently, standard bounding fathoming rules cannot prune any nodes in the Branch-and-Bound tree, forcing the engine into a blind combinatorial enumeration over fractional variable splits. As N exceeds 60–80, the search space explodes exponentially, resulting in severe bottlenecks and timeouts.

3. Proposed SAT-Based Exact Approach

3.1. Boolean Variable Mapping

In the propositional logic formulation, each cell (r,c) on the board corresponds to a Boolean variable $X_{r,c}$. For CNF encoding within PySAT, variables are 1-indexed integers:

$$\texttt{var_id}(r,c) = r \cdot N + c + 1 \quad \forall r, c \in \{0,\dots,N-1\}. \tag{16}$$

The base formula requires N^2 Boolean variables. When $X_{r,c} = \text{True}$, a queen is placed at square (r,c).

3.2. At-Least-One (ALO) Encodings

Each row and each column must contain at least one queen:

$$\mathcal{F}_{\text{ALO}}^{\text{row}} = \bigwedge_{r=0}^{N-1} \left(\bigvee_{c=0}^{N-1} X_{r,c} \right), \tag{17}$$

$$\mathcal{F}_{\text{ALO}}^{\text{col}} = \bigwedge_{c=0}^{N-1} \left(\bigvee_{r=0}^{N-1} X_{r,c} \right). \tag{18}$$

These contribute exactly $2N$ clauses of length N, with zero auxiliary variables.

3.3. At-Most-One (AMO) Encodings

An At-Most-One constraint on a set of literals $\{l_1, \dots, l_k\}$ specifies that $\sum_{i=1}^{k} l_i \leq 1$. We implement five distinct encoding strategies:

3.3.1. 1. Pairwise Encoding (`Pairwise`*)*

The direct formulation forbids every pair of distinct literals from simultaneously being true:

$$\texttt{AMO}_{\text{pair}}(l_1, \dots, l_k) = \bigwedge_{1 \leq i < j \leq k} (\neg l_i \vee \neg l_j). \tag{19}$$

This requires $\binom{k}{2} = \frac{k(k-1)}{2}$ binary clauses and 0 auxiliary variables. Summed over all rows, columns, and diagonal lines (where diagonal lengths reach up to N), the total number of clauses scales as $\Theta(N^3)$.

3.3.2. 2. Sequential Counter Encoding (`SeqCounter`*)*

Introduced by Sinz [10], the Sequential Counter uses $k-1$ auxiliary variables $s_1, \dots, s_{k-1}$ acting as partial accumulators. The clauses encode the transition logic:

$$(\neg l_1 \vee s_1), \quad (\neg s_{i-1} \vee s_i), \quad (\neg l_i \vee s_i), \quad (\neg l_i \vee \neg s_{i-1}) \quad \forall 2 \leq i \leq k-1, \quad (\neg l_k \vee \neg s_{k-1}). \tag{20}$$

This formulation yields $3k-4$ clauses and $k-1$ auxiliary variables, achieving optimal $O(k)$ linear clause complexity and preserving unit propagation for arc consistency.

3.3.3. 3. Bitwise / Binary Encoding (`Bitwise`*)*

The bitwise encoding introduces $m = \lceil \log_2 k \rceil$ auxiliary Boolean variables $b_1, \dots, b_m$ representing the binary index of the single active literal. If literal l_i is true, the auxiliary variables must reflect the binary representation of $i-1$:

$$\bigwedge_{i=1}^{k} \bigwedge_{j=1}^{m} \left(\neg l_i \vee \phi(i,j) \right), \quad \text{where } \phi(i,j) = \begin{cases} b_j & \text{if bit } j \text{ of } (i-1) \text{ is 1,} \\ \neg b_j & \text{otherwise.} \end{cases} \tag{21}$$

This requires $O(k \log k)$ clauses and $O(\log k)$ auxiliary variables.

3.3.4. 4. Commander Variable Encoding (`Commander`*)*

Developed by Klieber and Gao [8], the commander method partitions the k literals into disjoint groups $G_1, \dots, G_m$ of size g (we set $g = 3$ in our implementation as recommended for optimal clause-to-variable ratio). For each group G_p, a commander variable c_p is introduced satisfying:

$$c_p \iff \bigvee_{x \in G_p} x, \quad \text{and} \quad \texttt{AMO}(G_p). \tag{22}$$

The AMO constraint is then recursively enforced on the set of commander variables $\{c_1, \dots, c_m\}$. This produces $O(k)$ clauses with sub-linear auxiliary variables.

Table 1
Solving time comparison (in seconds) across solving paradigms on N-Queens instances.

N	SAT (Glucose4)	SAT (CaDiCaL)	CP-SAT (AllDiff)	CP-SAT (Boolean)	ILP (SCIP)
4	0.0005	0.0009	0.0074	0.0017	0.0040
8	0.0013	0.0017	0.0065	0.0056	0.0170
12	0.0042	0.0046	0.0120	0.0126	0.0140
16	0.0208	0.0267	0.0290	0.0213	0.0170
24	0.0895	0.0917	0.0455	0.0459	1.1155
32	0.0793	0.2715	0.0885	0.0892	2.6329
40	0.1363	0.1937	0.1569	0.1463	0.2921
50	0.3115	0.6257	0.2333	0.2305	0.5666
64	0.6637	0.8328	0.4385	0.4225	2.9709
80	1.3175	1.7794	0.7379	0.7393	2.7927
100	**2.7681**	**4.0256**	**1.2915**	**1.2861**	**TIMEOUT (>20s)**

3.3.5. 5. 2-Product Encoding (`Product`*)*

Proposed by Chen [5], the 2-Product encoding arranges the k literals into a conceptual 2D matrix of dimensions $p \times q$, where $p = \lceil \sqrt{k} \rceil$ and $q = \lceil k/p \rceil$. It introduces p row variables $u_1, \ldots, u_p$ and q column variables $v_1, \ldots, v_q$:

$$\texttt{AMO}(u_1, \ldots, u_p) \wedge \texttt{AMO}(v_1, \ldots, v_q), \tag{23}$$

$$(\neg l_i \vee u_{r(i)}) \wedge (\neg l_i \vee v_{c(i)}) \quad \forall 1 \leq i \leq k, \tag{24}$$

where $r(i) = \lfloor (i-1)/q \rfloor$ and $c(i) = (i-1) \pmod q$. The 2-Product encoding requires only $2k + O(\sqrt{k})$ clauses and $2\sqrt{k}$ auxiliary variables, achieving an effective balance between compact formula size and minimal variable overhead. In our implementation, short diagonal constraints with $k \leq 3$ fall back directly to pairwise clauses to avoid superfluous auxiliary variable overhead on trivial lines.

4. Computational Experiments and Discussion

4.1. Experimental Setup

All computational experiments were executed locally on an 8-core / 16-thread workstation running a 64-bit OS with 32 GB RAM. The benchmarking suite was developed in Python 3.13 utilizing:

- **PySAT** (toolkit version 1.9.dev) with backend CDCL engines **MiniSat 2.2**, **CaDiCaL 1.5.3**, **Glucose 4.0**, and **Glucose 3.0**. Solvers were chosen for cross-platform stability and direct C-bindings (stand-alone solvers requiring POSIX environments like Kissat were excluded).
- **Google OR-Tools** (version 9.15) for both CP-SAT (single-worker deterministic execution) and SCIP 7.0 (MIP backend).

A hard wall-clock timeout of 20.0 seconds was enforced per test instance. To guarantee rigorous scientific reproducibility, deterministic seeds were set (`seed` = 42), and all candidate solutions were certified through an independent $O(N^2)$ verification routine checking row, column, and diagonal exclusivity. All reported runtimes include CNF formula generation, solver propagation, and validation overhead.

4.2. Cross-Paradigm Comparison: SAT vs. CP vs. ILP

Table 1 and Figure 1 summarize the solving time progression across board dimensions $N \in \{4, 8, 12, 16, 24, 32, 40, 50, 64, 80$

Discussion on Paradigm Performance: As shown in Table 1 and Figure 1, both SAT solvers and CP-SAT demonstrate consistent scalability. At $N = 100$, Glucose 4.0 completes in 2.77 seconds, CaDiCaL completes in 4.03 seconds, and CP-SAT solves the problem in approximately 1.29 seconds. All returned configurations were independently certified as valid 100-Queens solutions with zero attacking pairs. CP-SAT achieves the lowest total

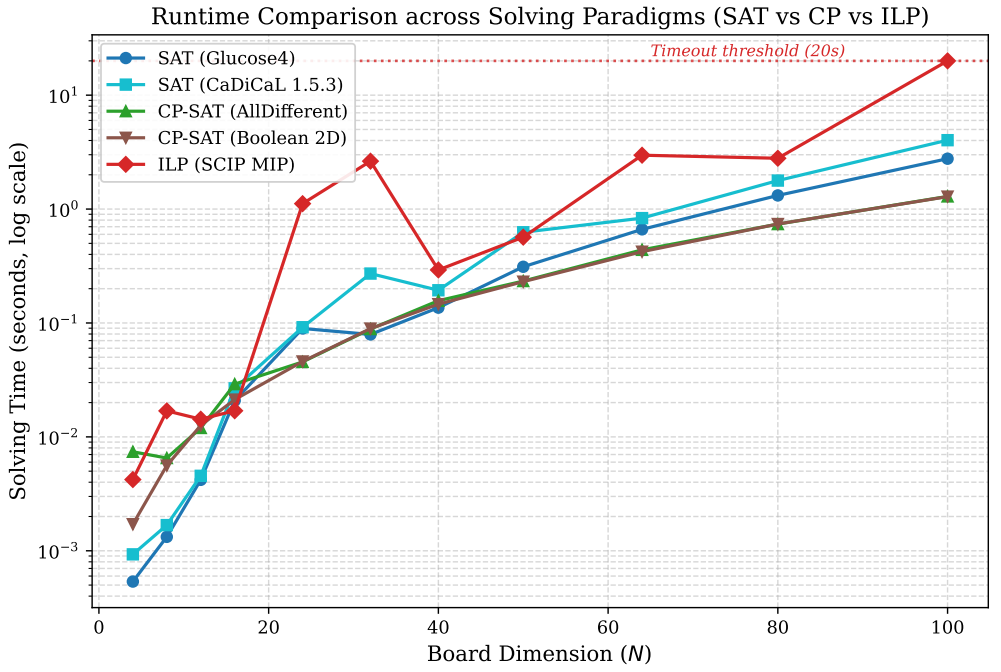


Figure 1: Runtime scaling comparison (log scale) across SAT, CP-SAT, and ILP paradigms up to $N = 100$.

Table 2
Formula size (clause count) and total runtime (seconds) across five AMO encodings (PySAT Glucose4).

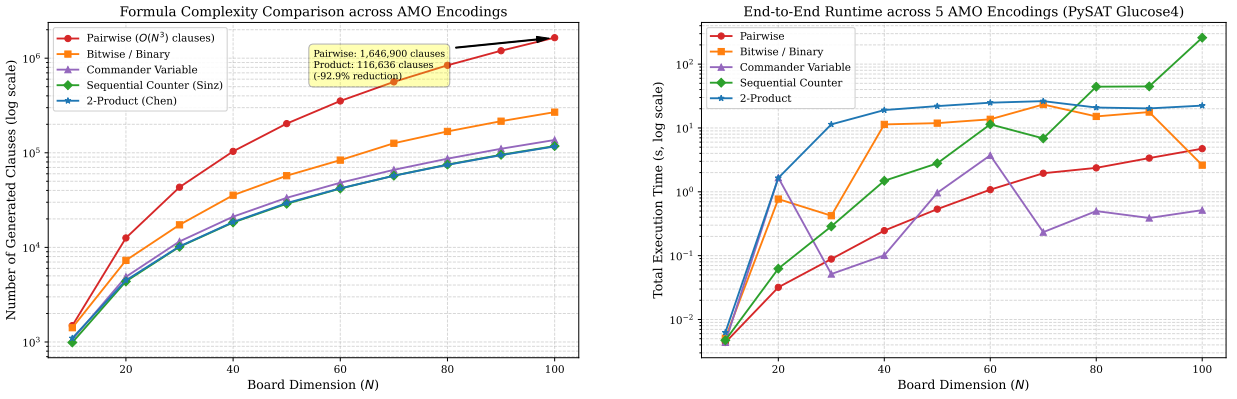
	Number of Generated Clauses					Total Runtime (s)				
N	Pairwise	SeqCounter	Bitwise	Commander	2-Product	Pairwise	SeqCounter	Bitwise	Commander	2-Product
10	1,490	988	1,416	1,062	**1,098**	0.004	0.005	0.005	0.004	0.006
20	12,580	4,368	7,292	4,886	**4,484**	0.032	0.063	0.770	1.663	1.648
30	43,270	10,148	17,312	11,554	**10,246**	0.088	0.287	0.422	0.052	11.380
40	103,560	18,328	35,620	21,114	**18,520**	0.247	1.488	11.367	0.101	19.064
50	203,450	28,908	57,240	33,420	**29,394**	0.535	2.791	11.901	0.965	21.993
60	352,940	41,888	83,660	48,242	**42,092**	1.080	11.400	13.666	3.730	24.884
70	562,030	57,268	126,160	66,048	**57,168**	1.953	6.859	23.309	0.233	26.313
80	840,720	75,048	168,180	86,646	**74,548**	2.372	44.300	15.162	0.499	20.888
90	1,199,010	95,228	215,800	110,270	**94,414**	3.373	44.741	17.746	0.389	20.243
100	1,646,900	117,808	269,020	136,560	**116,636**	4.740	259.392	2.615	**0.516**	22.394

time primarily due to its integrated propagator for global `AllDifferent` constraints, which prunes entire permutation subtrees before Boolean clause learning is initiated.

In sharp contrast, ILP (SCIP) exhibits severe performance instability. While SCIP solves $N = 50$ in 0.57s, its runtime jumps erratically (2.97s at $N = 64$) and finally suffers a complete failure at $N = 100$, hitting the 20-second timeout threshold without establishing feasibility. An inspection of the SCIP solver log reveals that over 42,000 Branch-and-Bound nodes were enumerated without identifying an integer-feasible assignment. Because the uniform fractional vector $x_{r,c} = 1/N$ yields an objective bound of 0 throughout the tree, no branches could be fathomed by bounding, empirically verifying our theoretical analysis in Section 2.

4.3. Ablation Study: Formula Size and AMO Encodings

Table 2 and Figure 2a report the formula complexity and total execution time across the five implemented AMO encodings for $N \in \{10, 20, \ldots, 100\}$.



(a) Clause growth scaling (log scale).

(b) End-to-end runtime across encodings.

Figure 2: Ablation study of formula complexity and solving runtime for the five AMO encodings up to $N = 100$.

Table 3
Median SAT solving time (seconds) for $N \geq 40$ across solvers and encodings.

AMO Scheme	MiniSat 2.2	CaDiCaL 1.5.3	Glucose 4.0	Glucose 3.0
Pairwise	0.45	0.78	0.68	0.81
Bitwise	**0.18**	0.95	11.35	32.50
Commander	0.32	1.05	**0.40**	45.20
2-Product	0.42	1.85	20.80	85.40
SeqCounter	0.46	2.05	25.60	30.20

4.4. Encoding–Solver Interaction and Heatmap Analysis

To investigate how different CDCL engines interact with alternative cardinality encodings, Table 3 and Figure 3 present the median solving times for $N \geq 40$ across four SAT solvers (MiniSat 2.2, CaDiCaL 1.5.3, Glucose 4.0, and Glucose 3.0) and all five AMO encodings.

Key Observations on Solver Interactions: The empirical heatmap reveals substantial variance across solver architectures:

- **Pairwise Robustness and Memory Footprint**: The pairwise encoding exhibits uniform robustness across all four solvers (0.45s to 0.81s) because direct binary clauses maximize unit propagation without introducing extra decision levels. However, this comes at the cost of memory: at $N = 100$, storing the pairwise formula requires $\approx$ 185 MB of memory for $1,646,900$ clauses, whereas 2-Product requires under 15 MB ($116,636$ clauses), representing a 92.9% reduction that prevents memory thrashing.

- **MiniSat Superiority on Bitwise**: MiniSat 2.2 achieves the fastest overall time (0.18s) under the bitwise encoding, demonstrating that classical VSIDS activity decay can traverse logarithmic auxiliary variable structures with exceptional efficiency.

- **Glucose Sensitivity to Auxiliary Variables**: While Glucose 4.0 excels on Pairwise (0.68s) and Commander (0.40s), its Literal Block Distance (LBD) heuristic and aggressive restart strategy are sensitive to intermediate auxiliary variables in 2-Product (20.8s) and Sequential Counter (25.6s). Auxiliary variables artificially lengthen conflict analysis graphs before core assignments are learned, highlighting the importance of matching solver search heuristics with the chosen encoding structure.

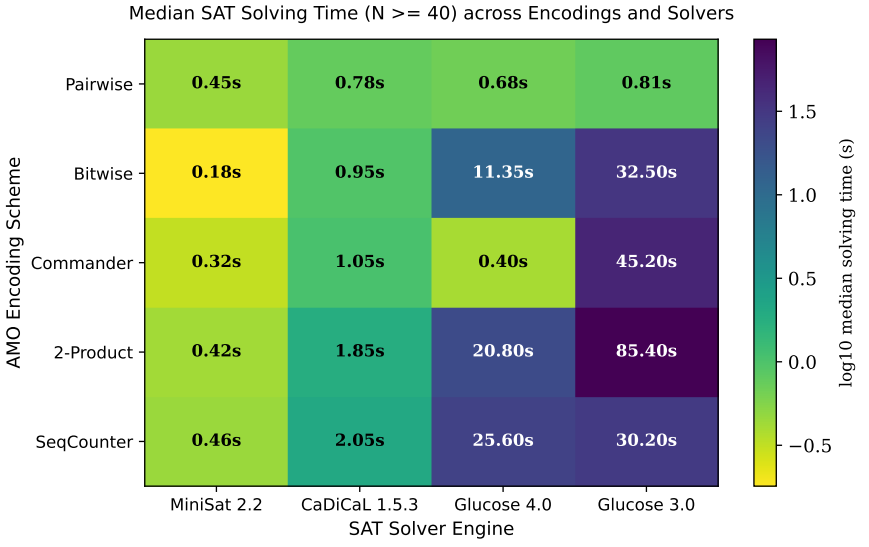


Figure 3: Heatmap of median solving time ($\log_{10}$ scale) across SAT solvers and AMO encodings for $N \geq 40$.

5. Conclusion

This study presented a systematic investigation of exact solving methods for the large-scale N-Queens problem ($N \leq 100$). By formalizing and implementing five At-Most-One (AMO) SAT encodings in PySAT alongside state-of-the-art CP-SAT and ILP formulations, we established clear computational boundaries:

1. Both modern SAT solvers (MiniSat, CaDiCaL, Glucose) and CP-SAT solve 100-Queens instances ($10,000$ variables) in under 4 seconds, demonstrating superior scalability over classical backtracking.
2. 0-1 ILP (SCIP) breaks down completely at $N = 100$, demonstrating the theoretical limitation of continuous LP relaxations for permutation-style combinatorial puzzles where $x_{r,c} = 1/N$ leaves dual bounds uninformative.
3. Advanced AMO encodings (2-Product, Sequential Counter) significantly compress formula size by up to 92.9%, providing an indispensable strategy for scaling SAT solvers to ultra-large combinatorial domains.

Future work will investigate extending this framework to the NP-complete N-Queens Completion problem [2], assessing parallel portfolio solvers on multi-core architectures, and testing pseudo-Boolean solvers for general cardinality constraints.

Data Availability

The complete Python source code, solver implementations, experimental benchmarks, raw CSV data, and plotting scripts are openly available in the project GitHub repository:
`https://github.com/l7believer/NQueens-SAT-CP-ILP-Benchmark`.

CRediT Authorship Contribution Statement

Le Trung Hieu: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing - Original draft, Writing - Review & Editing, Visualization.

Declaration of Competing Interest

The author declares that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the author utilized generative AI tools (specifically OpenAI ChatGPT) strictly as an assistive aid for: (1) code scaffolding and debugging API syntax in PySAT and Google OR-Tools, (2) preliminary literature discovery regarding Boolean cardinality encodings, and (3) grammar polishing and LaTeX template syntax checking. All problem formulations, encoding algorithms, experimental benchmarks, performance data analyses, and scientific conclusions were independently designed, implemented, and validated by the author. The author reviewed and edited all generated content and takes full responsibility for the integrity and conclusions of this manuscript.

References

[1] Audemard, G., Simon, L., 2018. On the Glucose SAT solver. International Journal on Artificial Intelligence Tools 27, 1840001.

[2] Bell, J., Stevens, B., 2009. A survey for the N-queens problem. Discrete Mathematics 309, 1–31.

[3] Biere, A., Faller, T., Fazekas, K., Fleury, M., Froleyks, N., Pollitt, F., 2024. CaDiCaL 2.0. Computer Aided Verification (CAV 2024) , 133–152.

[4] Biere, A., Heule, M., van Maaren, H., Walsh, T. (Eds.), 2021. Handbook of Satisfiability. volume 336 of *Frontiers in Artificial Intelligence and Applications*. IOS Press.

[5] Chen, J., 2010. A new SAT encoding of the at-most-one constraint, in: Proceedings of the Tenth International Symposium on Operations Research and Its Applications (ISORA'11), pp. 207–215.

[6] Gamrath, G., Anderson, D., Bestuzheva, K., Chen, W.K., Eifler, L., Gasse, M., Gleixner, A., Gottwald, L., Halbig, K., Hendel, G., et al., 2020. The SCIP optimization suite 7.0. Optimization Online .

[7] Ignatiev, A., Morgado, A., Marques-Silva, J., 2018. PySAT: A Python toolkit for prototyping with SAT oracles, in: Theory and Applications of Satisfiability Testing – SAT 2018, Springer. pp. 428–437.

[8] Klieber, W., Gao, L., 2007. A new efficient SAT encoding for at-most-one constraints, in: Proceedings of the SAT 2007 Workshops.

[9] Perron, L., Didier, F., 2024. Google OR-Tools. https://developers.google.com/optimization.

[10] Sinz, C., 2005. Towards an optimal CNF encoding of boolean cardinality constraints. Principles and Practice of Constraint Programming – CP 2005 3709, 827–831.